Le modèle mémoire de Python

Objets, références, mutabilité, copies, effets de bord, portée et structures de données. Chaque résultat est d'abord énoncé rigoureusement, puis expliqué « en clair ».

1. Objets et noms
2. Mutabilité
3. Mutation ou réaffectation
4. Égalité et identité
5. Copie superficielle et profonde
6. Fonctions et effets de bord
7. Portée des variables
8. Les quatre structures
9. Exercices corrigés
10. À savoir par cœur

---

1. Objets et noms

Définition 1.1 *(objet)*

Toute donnée manipulée par Python est un **objet**, stocké en mémoire. Un objet possède trois attributs :

• une **identité**, son adresse en mémoire, donnée par `id(x)`, fixe pendant toute sa vie ;
• un **type**, donné par `type(x)`, qui fixe ce qu'on peut faire avec lui ;
• une **valeur**, son contenu.

Définition 1.2 *(nom, référence)*

Une **variable** est un **nom** qui **référence** un objet. L'instruction `x = expr` évalue d'abord `expr`, ce qui produit un objet, puis lie le nom `x` à cet objet.

En clair : une variable n'est pas une boîte qui contient la valeur. C'est une **étiquette collée** sur un objet.

```
notes = [12, 15, 9]
copie = notes          # aucune copie : on colle une 2e étiquette

   notes ─┐
          ├──►  [12, 15, 9]     ← un seul objet
   copie ─┘
```

> **Proposition 1.3**
>
> `b = a` ne crée jamais d'objet. Après cette ligne, `a` et `b` référencent **le même objet**, donc `id(a) == id(b)`.

---

2. Mutabilité

> **Définition 2.1** *(mutable / immuable)*
>
> Un objet est **mutable** si sa valeur peut changer sans que son identité change. Sinon, il est **immuable**.
>
> **En clair :** on peut modifier un objet mutable « sur place ». Un objet immuable, jamais : pour « changer » sa valeur, Python fabrique un nouvel objet.

IMMUABLE	MUTABLE
`int` · `float` · `bool`	`list`
`str`	`dict`
`tuple`	`set`

> **Corollaire 2.2**
>
> Le partage d'un objet entre plusieurs noms n'a de conséquence visible que si l'objet est **mutable**. Pour un objet immuable, il est sans danger.

3. Mutation ou réaffectation

C'est la distinction centrale de la fiche. Deux opérations se ressemblent mais n'ont rien à voir.

	MUTATION	RÉAFFECTATION
Syntaxe	`a.append(x)`, `a[i] = x`, `d[k] = v`, `a += [x]` (sur une liste)	`a = …`
Agit sur	l'**objet**	le **nom**
`id(a)`	inchangé	change
Les autres noms	**voient** le changement	ne sont **pas** touchés

```
# Mutation                    # Réaffectation
a = [1, 2]                    a = [1, 2]
b = a                         b = a
a.append(3)                   a = [7, 8]
print(b)  # [1, 2, 3]         print(b)  # [1, 2]
```

Piège 3.1 *(`a = a + [x]` et `a += [x]`)*

Pour une liste, `a = a + [x]` construit une **nouvelle** liste (réaffectation), alors que `a += [x]` **modifie** la liste existante (mutation). Les autres étiquettes ne verront donc le changement que dans le second cas.

4. Égalité et identité

Définition 4.1

`a == b` teste l'**égalité des valeurs**. `a is b` teste l'**identité**, c'est-à-dire `id(a) == id(b)`.

En clair : deux jumeaux sont `==` (ils se ressemblent) mais pas `is` (ce sont deux personnes).

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
a == b   # True   même contenu
a is b   # False  deux objets distincts
a is c   # True   le même objet
```

> **Règle d'usage**
>
> On compare des valeurs avec `==`. On réserve `is` à `None` : on écrit `if x is None:`.

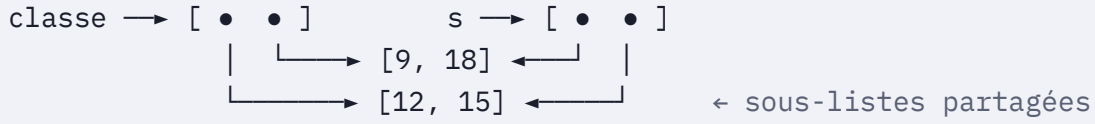
---

5. Copie superficielle et profonde

> **Définition 5.1** *(copie superficielle)*
>
> Une **copie superficielle** (*shallow copy*) crée un nouveau conteneur, mais ses éléments sont **les mêmes objets** que ceux de l'original.
>
> Les syntaxes équivalentes sont `a.copy()`, `list(a)`, `a[:]` et `copy.copy(a)`.

> **Définition 5.2** *(copie profonde)*
>
> Une **copie profonde** (*deep copy*) copie récursivement le conteneur **et tout ce qu'il contient**. On l'obtient avec `from copy import deepcopy`, puis `deepcopy(a)`.

```
classe = [[12, 15], [9, 18]]
s = classe.copy()
s[0].append(20)
print(classe)   # [[12, 15, 20], [9, 18]]  l'original est touché !
```




```
from copy import deepcopy
d = deepcopy(classe)
d[0].append(20)
print(classe)   # [[12, 15], [9, 18]]  intact
```

L'OBJET CONTIENT…	COPIE SUFFISANTE
uniquement des immuables (nombres, textes)	`.copy()`
des mutables (listes de listes, dict de listes…)	`deepcopy()`

6. Fonctions et effets de bord

> **Proposition 6.1** *(passage des arguments)*
>
> Lors d'un appel `f(x)`, le paramètre de `f` devient une **nouvelle étiquette sur le même objet** que `x`. Python ne fait aucune copie. On dit que les arguments sont passés « par partage ».

> **Définition 6.2** *(effet de bord)*
>
> Une fonction a un **effet de bord** si elle modifie un état extérieur à elle-même : un argument mutable, une variable globale, un fichier…

```
# ✗ Version fautive : abîme l'argument
def normaliser_notes(notes):
    for i in range(len(notes)):
        notes[i] = notes[i] / 20
    return notes

brutes = [12, 15, 9, 18]
norm = normaliser_notes(brutes)
print(brutes)   # [0.6, 0.75, 0.45, 0.9]  données d'origine perdues

# ✓ Correction 1 : travailler sur une copie
def normaliser_notes(notes):
    notes = notes.copy()
    for i in range(len(notes)):
        notes[i] = notes[i] / 20
    return notes

# ✓ Correction 2 : construire une nouvelle liste
def normaliser_notes(notes):
    resultat = []
    for n in notes:
        resultat.append(n / 20)
    return resultat
```

> **Règle de conception**
>
> Une fonction de traitement de données **ne modifie pas ses entrées**. Elle **renvoie** un nouvel objet. Si une modification sur place est voulue, le nom de la fonction l'indique et elle renvoie `None`, comme `list.sort()`.

Piège 6.3 *(argument par défaut mutable, approfondi à 13 h)*

```
def ajouter(x, liste=[]):   # la liste [] est créée UNE fois, à la définiti
    liste.append(x)
    return liste
ajouter(1)  # [1]
ajouter(2)  # [1, 2]  !
```

Correction : `def ajouter(x, liste=None):` puis, dans la fonction, `if liste is None: liste = []`.

Dans le métier

Un pipeline ML enchaîne des fonctions : charger, nettoyer, normaliser, entraîner. Si une étape modifie ses entrées en silence, les données du jeu de test peuvent être contaminées. Le modèle semble alors excellent, mais il est faux, et **aucune erreur ne s'affiche**.

---

7. Portée des variables

Définition 7.1 *(portée, règle LEGB)*

La **portée** d'un nom est la zone du code où il est visible. Pour trouver un nom, Python cherche dans cet ordre :

Local (dans la fonction) → **E**nglobant (fonction parente) → **G**lobal (le fichier) → **B**uilt-in (`print` , `len` ...).

```
def calculer():
    total = 50          # nom LOCAL : il n'existe que pendant l'appel
    return total

calculer()
print(total)            # NameError: name 'total' is not defined

resultat = calculer()   # on récupère la valeur grâce à return
```

Proposition 7.2

Affecter un nom dans une fonction le rend **local**. Pour faire sortir une valeur d'une fonction, on utilise `return` , pas `global` .

8. Les quatre structures de données

TYPE	SYNTAXE	ORDONNÉ	MUTABLE	DOUBLONS	`x in …`	USAGE ML
list	`[1, 2]`	oui	oui	oui	O(n)	prédictions, scores
tuple	`(1, 2)`	oui	non	oui	O(n)	forme d'un tableau `(224, 224)`
dict	`{"k": v}`	ordre d'insertion	oui	clés uniques	O(1)	hyperparamètres, comptages
set	`{1, 2}`	non	oui	non	O(1)	étiquettes distinctes

O(n) : Python doit parcourir les éléments un par un. **O(1)** : il trouve directement, quelle que soit la taille. Big O est étudié en détail cet après-midi.

```
notes = [12, 15, 9]           ; notes[0]            # 12, on indexe à partir de 0
taille = (224, 224)           ; taille[0] = 10      # TypeError : un tuple est im
config = {"lr": 0.001}        ; config["epochs"] = 10
themes = {"forces", "énergie", "forces"}  # {'forces', 'énergie'}
{}                                        # attention : c'est un dict vide ! set
```

9. Exercices corrigés

Méthode imposée : écrire sa prédiction **et sa justification** avant de déplier le corrigé.

Exercice 1 ★ application directe

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Corrigé. `[1, 2, 3, 4]`. Après `b = a`, les deux noms référencent la même liste (Prop. 1.3). `append` est une mutation : `a` voit le changement.

Exercice 2 ★ immuable

```
x = "chat"
y = x
y = y + "s"
print(x, y)
```

Corrigé. `chat chats`. Un `str` est immuable : `y + "s"` crée un nouvel objet, et `y = …` déplace l'étiquette `y` sur lui. `x` n'est pas touché.

Exercice 3 ★ == et is

```
a = [1, 2, 3]; b = [1, 2, 3]; c = a
print(a == b, a is b, a is c)
```

Corrigé. `True False True`. Voir Définition 4.1.

Exercice 4 ★★ le piège

```
b = a = [0]
a = a + [1]
b += [2]
print(a, b)
```

Corrigé. `[0, 1] [0, 2]`. `a = a + [1]` crée une nouvelle liste `[0, 1]` et y déplace `a`. `b` reste sur l'ancienne liste `[0]`, que `b += [2]` modifie sur place (Piège 3.1).

Exercice 5 ★★ copie superficielle

```
m = [[1, 2], [3, 4]]
n = m.copy()
n.append([5, 6])
n[0][0] = 99
print(m)
```

Corrigé. `[[99, 2], [3, 4]]`. `n.append` modifie le nouveau conteneur, donc `m` ne reçoit pas `[5, 6]`. En revanche, `n[0]` **est** la même sous-liste que `m[0]` (Déf. 5.1) : le 99 apparaît dans `m`.

Exercice 6 ★★ effet de bord · exercice officiel du jour

Expliquer pourquoi `normaliser_notes` (§6) modifie la liste d'origine, puis corriger.

Corrigé. Le paramètre `notes` est une étiquette sur la même liste que `brutes` (Prop. 6.1). `notes[i] = …` est une mutation, donc `brutes` est modifiée : c'est un effet de bord (Déf. 6.2). On corrige en travaillant sur une copie (`notes = notes.copy()`) ou en construisant une nouvelle liste (voir les deux corrections du §6).

Exercice 7 ★★★ synthèse

```
def f(x, L):
    x = x + 1
    L.append(x)
    L = [0]
    return L

n = 5
M = [1]
R = f(n, M)
print(n, M, R)
```

Corrigé. `5 [1, 6] [0]`.

- `x = x + 1` réaffecte le nom local `x`. Un `int` est immuable, donc `n` vaut toujours 5.
- `L.append(6)` mute la liste partagée avec `M`, qui devient `[1, 6]`.
- `L = [0]` déplace seulement l'étiquette locale `L`. `M` n'est pas touchée, et c'est cette nouvelle liste `[0]` qui est renvoyée.

10. À savoir par cœur

1. Une variable est un **nom** qui référence un **objet**. L'objet a une identité, un type et une valeur.
2. `b = a` ne copie rien : deux noms, un seul objet.
3. `list`, `dict` et `set` sont mutables. `int`, `float`, `bool`, `str` et `tuple` sont immuables.
4. Une **mutation** change l'objet, et tous les noms la voient. Une **réaffectation** `=` déplace un seul nom.
5. `==` compare les valeurs. `is` compare les identités, et on l'utilise pour `None`.
6. `.copy()` copie le premier niveau seulement. `deepcopy()` copie tout.
7. Une fonction reçoit l'objet lui-même, pas une copie. Elle ne doit pas muter ses entrées : elle **renvoie** un nouvel objet.
8. Jamais de valeur par défaut mutable : on écrit `liste=None`.
9. Un nom affecté dans une fonction est local. On sort une valeur avec `return`.
10. La recherche `in` coûte O(n) dans une liste et O(1) dans un set ou un dict.
